Jetpack Compose: Advanced UI

Muhammad Naveed

November 19, 2025

Chapter 24: Overview of Slot APIs

Exploring flexible and reusable composable functions.

What is a Slot API?

Simple Explanation

The Slot API is a pattern in Jetpack Compose that allows you to create flexible, reusable components by letting the caller decide what content goes into specific "slots" of your composable.

Think of it as creating a template with empty spaces that can be filled with different content.

```
[EMPTY SPACE]

← This is a SLOT
  You decide what picture goes here
```

What is a Slot API? The Problem

Without Slot API:

- You'd need to create multiple versions of similar components (e.g., ButtonWithIcon, ButtonWithText, ButtonWithBoth)
- Violates the Open/Closed Principle (open for extension, closed for modification)

With Slot API:

- Single, flexible component that accepts different content.
- Clean separation of layout logic and content.

Chapter 25: Slot API Tutorial

A practical example of building a dynamic UI.

Tutorial: Project Goal

What We're Building

A dynamic UI application that demonstrates the Slot API pattern through:

- Title Display: Switchable between Image or Text
- Progress Indicator: Switchable between Linear or Circular
- Two Checkboxes: Control what content appears in each slot

Tutorial: Composable Structure

MainScreen

The "smart" composable that holds the state and event handlers.

ScreenContent

The "dumb" template that provides the Slot API and defines the layout.

CheckBoxes

A reusable composable to display the controls and report clicks.

Tutorial: The 'ScreenContent' Slot API

Slots Definition:

- @Composable () -> Unit = composable lambda parameter
- Named descriptively: titleContent, progressContent
- Optional parameters would use nullable type: (@Composable () -> Unit)?

Responsibilities:

- ScreenContent owns: Layout structure and arrangement
- ScreenContent doesn't know: What specific content fills slots
- Caller decides: Which composables to inject into slots

```kotlin
@Composable
fun ScreenContent(
    linearSelected: Boolean,
    imageSelected: Boolean,
    onTitleClick: (Boolean) -> Unit,
    onLinearClick: (Boolean) -> Unit,
    titleContent: @Composable () -> Unit,      // SLOT 1
    progressContent: @Composable () -> Unit    // SLOT 2
) {
    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.SpaceBetween
    ) {
        titleContent()      // Invoke title slot
        progressContent()   // Invoke progress slot
        CheckBoxes(linearSelected, imageSelected,
                   onTitleClick, onLinearClick)
    }
}
```

Tutorial: Connecting State to Slots

State-to-Slot Flow

User Interaction → State Change → Slot Content Updates

Checkbox State	**Title Slot Show**	**Progress Slot Show**
Both Checked	TitleImage	LinearProgressIndicator
Both Unchecked	Text	CircularProgressIndicator
Image Checked Linear Unchecked	Title Image	CircularProgressIndicator
Image Unchecked Linear Checked	Text	LinearProgressIndicator

Chapter 27: Annotated Strings

Styling individual parts of a text.

Beyond Basic Text Styling

The Problem

- Applies ONE style to the ENTIRE string
- Cannot mix bold + italic in same text
- Cannot have multiple colors in one string
- Limited flexibility for rich text formatting

```
Text(
    text = "Hello World",
    fontWeight = FontWeight.Bold,
    color = Color.Blue
)
```

Beyond Basic Text Styling - Continued

The Solution: Annotated Strings

What Are Annotated Strings?

- Allow text to be divided into multiple sections
- Each section can have its own independent style
- Created using buildAnnotatedString function
- Combine sections with append() calls

SpanStyle: Character-Level Styling

SpanStyle

Use buildAnnotatedString and withStyle to apply a SpanStyle.

SpanStyle controls character properties:

- color
- fontSize
- fontWeight
- fontStyle (Italic, etc.)
- textDecoration (Underline, etc.)

```
buildAnnotatedString {
    withStyle(style = SpanStyle(/* style settings */)) {
        append(/* text string */)
    }
    withStyle(style = SpanStyle(/* style settings */)) {
        append(/* more text */)
    }
}
```

ParagraphStyle: Block-Level Styling

ParagraphStyle

ParagraphStyle applies styles to an entire paragraph block within an annotated string.

It controls block-level properties:

- textAlign (Start, End, Center)
- lineHeight
- textIndent (e.g., firstLine, restLine)

```
buildAnnotatedString {
    withStyle(style = ParagraphStyle(/* style settings */)) {
        append(/* paragraph text */)
    }
    withStyle(style = ParagraphStyle(/* style settings */)) {
        append(/* more paragraph text */)
    }
}
```

BrushStyle: Advanced Text Effects

BrushStyle

For even more advanced styling, you can apply a Brush to the text foreground.

This allows you to fill text with gradients (linear, radial) or even an image.

The brush is applied as part of a SpanStyle or TextStyle.

```
val myColors = listOf(/* color list */)

Text(
    text = "text here",
    style = TextStyle(
        brush = Brush.linearGradient(
            colors = myColors
        )
    )
)
```

Questions?

Thank you for your attention.